

Step into the World of Code

Syllabus Overview

COURSE INFORMATION:

Course Title:	Web Development
Target Audience:	Students who have passed grade 10
Level:	Beginner to Intermediate
Duration:	50 Classes, 2 Hour Each
Mode of Delivery:	Online or in-person, hands-on learning.
Prerequisites:	Basic understanding of Computer and its terms.

Teaching Approach:

- ♦ Interactive lessons with step-by-step guidance.
- ♦ Project-based learning to reinforce concepts.
- ♦ Hands-on activities and real-world case studies.
- ♦ Collaboration and peer feedback to enhance understanding.

Learning Objectives :

By the end of this course, students will:

- ♦ Understand the fundamentals of UI/UX design and its importance in web development.
- ♦ Gain proficiency in HTML for building web page structures.
- ♦ Learn CSS for styling and creating responsive web designs.
- ♦ Master Tailwind CSS to efficiently design layouts and components.
- ♦ Develop interactive web pages using JavaScript.
- ♦ Build a personal portfolio website with design, HTML structure, and interactivity.
- ♦ Learn how to deploy web projects and ensure they are responsive and optimized.
- ♦ Create a final web development project showcasing their skills.

UNIT 1 INTRODUCTION TO UI/UX DESIGN

Class 1: Introduction to AI and Its Applications

Goal: Understand the fundamental differences between UI and UX, and learn the importance of UI/UX in web development.

Activity:

- ◆ Introduce UI (User Interface) and UX (User Experience) design.
- ◆ Discuss key design principles: contrast, alignment, hierarchy, balance, and proximity.
- ◆ Explore the difference between wireframes, mockups, and prototypes, and understand when to use each.

Class 2: Figma Introduction

Goal: Familiarize with the Figma interface and basic design tools.

Activity:

- ◆ Overview of the Figma workspace, tools, and functionality.
- ◆ Create and organize frames and layers in Figma.
- ◆ Practice creating shapes, text elements, and applying basic formatting.

Class 3: Designing with Figma

Goal: Learn how to create basic wireframes and design user flows for a simple page or app.

Activity:

- ◆ Build wireframes for a website or app layout.
- ◆ Structure UI for easy navigation and a smooth user experience.
- ◆ Map out user flows to visualize the user's journey through the design.

Class 4: Creating High-Fidelity Mockups

Goal: Learn how to transform wireframes into detailed, high-fidelity mockups in Figma.

Activity:

- ◆ Convert wireframes into polished designs by adding colors, fonts, and images.
- ◆ Organize and use design systems and reusable components for consistency across the design.

Class 5: Interactive Prototypes in Figma

Goal: Learn to create interactive prototypes using Figma.

Activity:

- ◆ Convert static designs into clickable prototypes.
- ◆ Set up interactive elements such as links, buttons, and transitions.
- ◆ Preview prototypes to simulate user interactions.

Class 6: Usability Testing

Goal: Understand how to conduct usability tests and gather feedback to improve designs.

Activity:

- ♦ Run basic usability tests with users and observe their interactions.
- ♦ Collect meaningful feedback to refine and improve the designs.
- ♦ Learn how to iterate on designs based on usability feedback.

Class 7: Figma Review and Q&A

Goal: Review the designs created and troubleshoot any issues.

Activity:

- ♦ Analyze the designs created during the week, focusing on UI/UX principles.
- ♦ Troubleshoot and resolve common design issues in Figma.
- ♦ Address any questions or doubts about the tools and techniques used.

UNIT 2 INTRODUCTION TO HTML

Class 8: Introduction to HTML

Goal: Learn the basic structure of HTML and its role in web development.

Activity:

- ♦ Install VSCode and set up the environment.
- ♦ Understand the basic structure of an HTML document (DOCTYPE, html, head, body).
- ♦ Learn the use of basic tags such as headings, paragraphs, and links.

Class 9: Working with Text, Links, and Images

Goal: Learn to format text, add links, and embed images in HTML.

Activity:

- ♦ Use tags like <p>, <h1>, <h2>, and others to format text.
- ♦ Add and format images using the tag.
- ♦ Create hyperlinks using the <a> tag.

Class 10: Tables in HTML

Goal: Learn file operations and pattern matching.

Activity:

- ♦ Build tables using <table>, <tr>, <th>, and <td> tags.
- ♦ Practice formatting tables to improve readability.

Class 11: Forms in HTML

Goal: Learn to build forms with various input elements in HTML.

Activity:

- ♦ Create forms using `<form>`, `<input>`, `<button>`, and other form elements.
- ♦ Learn about different input types: text, checkbox, radio, and dropdowns.

Class 12: Accessibility in HTML

Goal: Understand how to write accessible HTML and improve the user experience for all.

Activity:

- ♦ Learn about semantic HTML tags like `<header>`, `<footer>`, `<section>`, and `<article>`.
- ♦ Practice adding alt text for images to improve accessibility.

Class 13: Advanced HTML Features

Goal: Learn advanced HTML techniques, such as SEO and embedding media.

Activity:

- ♦ Use `<meta>` tags for SEO and social media sharing.
- ♦ Embed audio and video content using `<audio>` and `<video>` tags.
- ♦ Learn to use `<iframe>` for embedding external content.

Class 14: HTML Review and Q&A

Goal: Review key HTML concepts and address any questions.

Activity:

- ♦ Recap all HTML topics covered during the week.
- ♦ Troubleshoot common issues and clarify doubts about HTML coding.

UNIT 3 INTRODUCTION TO CSS

Class 15: Introduction to CSS

Goal: Learn the basics of CSS and how to apply styles to HTML elements.

Activity:

- ♦ Introduction to CSS syntax: selectors, properties, and values.
- ♦ Learn to apply CSS to HTML elements using inline, internal, and external styles.

Class 16: Styling Text and Layout Basics

Goal: Learn how to style text and understand the box model for layout.

Activity:

- ♦ Style fonts, colors, and backgrounds using CSS.
- ♦ Understand the box model: margins, padding, borders, and content areas.

Class 17: CSS Positioning

Goal: Learn the different types of CSS positioning and how to use them.

Activity:

- ◆ Understand the difference between relative, absolute, and fixed positioning.
- ◆ Practice using z-index and stacking contexts for positioning elements.

Class 18: Flexbox Layout

Goal: Learn how to create flexible layouts using Flexbox.

Activity:

- ◆ Understand flex containers and items.
- ◆ Use justify-content and align-items to align and distribute space in layouts.

Class 19: CSS Grid Basics

Goal: Learn how to create grid-based layouts in CSS.

Activity:

- ◆ Define rows, columns, and grid areas to structure your layout.
- ◆ Build a simple grid layout for a website.

Class 20: Responsive Web Design with CSS

Goal: Understand the concept of responsive design and how to implement it using CSS

Activity:

- ◆ Learn to use @media queries for responsive layouts.
- ◆ Implement a mobile-first approach for web design.

Class 21: CSS Review and Q&A

Goal: Review key CSS topics and troubleshoot any issues.

Activity:

- ◆ Recap Flexbox, Grid, and media queries.
- ◆ Address any CSS-related problems and assist with debugging.

UNIT 4 INTRODUCTION TO TAILWIND

Class 22: Introduction to Tailwind CSS

Goal: Get introduced to Tailwind CSS and learn how to set it up in a project.

Activity:

- ◆ Install and set up Tailwind CSS.
- ◆ Learn to use utility classes for styling elements.

Class 23: Styling with Tailwind

Goal: Learn how to apply Tailwind utility classes to style your webpage.

Activity:

- ◆ Style elements with Tailwind's utility classes for spacing, colors, and typography.
- ◆ Customize Tailwind configuration for custom themes.

Class 24: Building Layouts with Tailwind

Goal: Learn how to build responsive layouts using Tailwind CSS.

Activity:

- ◆ Use Tailwind's flex and grid utilities to structure responsive layouts.

Class 25: Tailwind Forms & Components

Goal: Learn to create and style forms and UI components with Tailwind.

Activity:

- ◆ Build forms and buttons using Tailwind's classes.
- ◆ Create reusable components like cards, modals, and navigation bars.

Class 26: Tailwind Layout Techniques

Goal: Refine your layout skills using Tailwind CSS utilities.

Activity:

- ◆ Create complex, responsive layouts using Tailwind's flex and grid systems.
- ◆ Use breakpoints for designing adaptive layouts.

Class 27: Tailwind Review and Q&A

Goal: Review the core concepts of Tailwind CSS and address any challenges.

Activity:

- ◆ Troubleshoot common issues with Tailwind.
- ◆ Address questions about the layout system, customization, and utility classes.

Class 28: Building a Small Project with Tailwind

Goal: Apply Tailwind CSS skills to build a small project.

Activity:

- ◆ Design a simple webpage using the utility-first approach of Tailwind.

UNIT 5 INTRODUCTION TO JAVASCRIPT (JS)

Class 29: Introduction to JavaScript

Goal: Learn the basics of JavaScript and its role in web development.

Activity:

- ◆ Introduction to variables, data types, and operators in JavaScript.
- ◆ Learn basic operations with numbers, strings, and booleans.

Class 30: Functions and Control Flow

Goal: Understand how to write functions and control flow in JavaScript.

Activity:

- ◆ Define and call functions.
- ◆ Write conditional statements (if/else) and loops (for/while).

Class 31: DOM Manipulation

Goal: Learn how to manipulate the Document Object Model (DOM) using JavaScript

Activity:

- ◆ Select elements using JavaScript (querySelector, getElementById).
- ◆ Change content and styles dynamically using JavaScript.

Class 32: Event Listeners and Interaction

Goal: Learn how to make interactive web pages with event listeners.

Activity:

- ◆ Add event listeners to handle user interactions like clicks and keypresses.
- ◆ Validate forms using JavaScript.

Class 33: Arrays and Loops

Goal: Understand arrays and how to loop through them

Activity:

- ◆ Create and manipulate arrays in JavaScript.
- ◆ Use loops to iterate over array items and perform actions.

Class 34: JavaScript Debugging

Goal: Learn how to debug JavaScript and write maintainable code.

Activity:

- ◆ Use browser developer tools for debugging JavaScript errors.
- ◆ Learn best practices for writing clean and efficient code.

Class 35: JavaScript Review and Q&A

Goal: Review JavaScript concepts and troubleshoot any issues.

Activity:

- ◆ Recap key JavaScript topics and answer questions.
- ◆ Help students fix common problems and improve their code.

Class 36: Portfolio Design in Figma

Goal: Plan and design a personal portfolio in Figma.

Activity:

- ◆ Create wireframes and high-fidelity designs for the portfolio.
- ◆ Organize sections like about, projects, and contact information.

Class 37: HTML Structure of Portfolio

Goal: Learn to create the HTML structure for the portfolio.

Activity:

- ◆ Build the basic structure of the portfolio: header, footer, and main content sections

Class 38: Styling Portfolio with Tailwind CSS

Goal: Apply Tailwind CSS to style the portfolio.

Activity:

- ◆ Use Tailwind's utility classes to style the portfolio layout according to the design

Class 39: Responsive Design with Tailwind

Goal: Make the portfolio responsive for different screen sizes.

Activity:

- ◆ Use Tailwind's responsive utilities to ensure the portfolio is mobile-friendly.

Class 40: JavaScript in Portfolio

Goal: Add interactivity to the portfolio.

Activity:

- ◆ Implement features like smooth scrolling and dynamic content with JavaScript.

Class 41: Testing and Debugging Portfolio

Goal: Test the portfolio and ensure it's working across different browsers.

Activity:

- ◆ Perform cross-browser testing and optimize performance.
- ◆ Troubleshoot any issues that arise.

Class 42: Final Touches on Portfolio

Goal: Refine and finalize the portfolio.

Activity:

- ◆ Make design tweaks based on feedback and personal preferences.

Class 43: Deployment and Hosting

Goal: Learn how to deploy the portfolio online.

Activity:

- ◆ Deploy the portfolio using platforms like GitHub Pages, Netlify, or Vercel.

Class 44: Portfolio Review and Feedback

Goal: Present the final portfolio and receive feedback.

Activity:

- ◆ Each student presents their portfolio and explains the process.
- ◆ Peer and instructor feedback to improve the projects.

Class 45: Project Setup & Homepage Layout

Goal: Set up the project files and design the homepage layout.

Activity:

- ◆ Overview of the project and introduction to the free movie API.
- ◆ Set up project files (HTML, CSS, JavaScript).
- ◆ Design the homepage structure (header, banner, movie sections).
- ◆ Apply basic CSS for layout and responsiveness.

Class 46: Fetching Movie Data from API

Goal: Learn how to fetch and display movie data using JavaScript.

Activity:

- ◆ Use the free movie API to fetch data (e.g., using fetch or axios).
- ◆ Display movies dynamically in the homepage layout.
- ◆ Learn to use JavaScript promises and asynchronous code.

Class 47: Building Movie Sections

Goal: Build interactive sections for movies like trending, popular, and upcoming.

Activity:

- ◆ Design and implement movie sections (e.g., trending, popular).
- ◆ Display movie posters, titles, and descriptions.
- ◆ Add JavaScript functionality to filter or paginate through movies.

Class 48: Adding Details and Interactivity

Goal: Add movie details pages and interactivity.

Activity:

- ♦ Create individual movie pages to display detailed information.
- ♦ Implement JavaScript for modal pop-ups and interactive features (e.g., trailers).

Class 49: Styling & Responsiveness

Goal: Ensure the Netflix clone is fully responsive and polished.

Activity:

- ♦ Apply advanced CSS and Tailwind for styling.
- ♦ Test and tweak responsiveness for various devices.)

Class 50: Final Touches and Deployment

Goal: Finish the project and deploy it online.

Activity:

- ♦ Perform final tests and bug fixes.
- ♦ Deploy the Netflix clone on a platform like Netlify or GitHub Pages.